

Lab 4.5: 基于 INT8 张量核的 FP64 GEMM 模拟

3240104505 查泮翊 (Grapesea)

1 理论推导与环境配置

1.1 算法原理

1.1.1 单个元素的逐级量化

1.1.2 矩阵乘法展开

1.2 初始实现与显存溢出

2 优化过程

2.1 融合多级量化

2.2 复用 INT32 scratch

2.3 workspace 缓存

2.4 pair 裁剪实验

3 最终 fast path

4 显存流量分析

1 理论推导与环境配置

本实验要求计算列主序矩阵乘法

$$C = AB, \quad A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}.$$

输入和输出都是 FP64，但实际乘法尽量交给 H800 的 INT8 Tensor Core. 核心想法是把一个 FP64 数拆成若干个带 scale 的 INT8 数，再把这些分量的乘积累加回来. 这样既可以利用 INT8 的高吞吐，又能通过增加 split 数恢复精度.

1.1 算法原理

1.1.1 单个元素的逐级量化

设当前需要分解的元素为 x . 对一个矩阵，先用所有元素的最大绝对值确定统一的初始比例尺：

$$s_0 = \frac{X_{\max}}{127}, \quad X_{\max} = \max_{i,j} |X_{ij}|.$$

统一 scale 很重要：如果每个元素使用不同 scale，后面的矩阵乘法就不能只乘一个标量，而需要额外的逐元素乘法，数据流会复杂很多.

令 $r_0 = x$ ，第 i 级量化得到整数 q_i ，并留下残差 r_{i+1} ：

$$q_i = \text{round}\left(\frac{r_i}{s_i}\right), \quad r_{i+1} = r_i - q_i s_i.$$

代码中把下一层比例尺设为

$$s_{i+1} = \frac{s_i}{254}.$$

这个常数可以直接从范围推出来. 四舍五入以后有 $|r_{i+1}| \leq s_i/2$. 为了保证下一层仍然落在 INT8 范围内，需要

$$\left| \frac{r_{i+1}}{s_{i+1}} \right| \leq 127.$$

把前一个误差上界代入，就得到

$$s_{i+1} \geq \frac{s_i/2}{127} = \frac{s_i}{254}.$$

所以 254 是不溢出时允许的最大缩减率. 经过 S 个 split 后，元素可以写成

$$x \approx \sum_{i=0}^{S-1} q_i s_i, \quad q_i \in [-127, 127].$$

在理想实数运算下，最后一级残差满足

$$|r_S| \leq \frac{s_0}{2 \cdot 254^{S-1}} = \frac{X_{\max}}{254^S}.$$

这个界只描述量化误差，不包括 FP64 舍入和矩阵乘法中的误差.

1.1.2 矩阵乘法展开

分别对 A 和 B 做分解：

$$A \approx \sum_{i=0}^{S-1} s_i^A A_q^{(i)}, \quad B \approx \sum_{j=0}^{S-1} s_j^B B_q^{(j)}.$$

代入 $C = AB$, 得到

$$C \approx \sum_{i=0}^{S-1} \sum_{j=0}^{S-1} s_i^A s_j^B \left(A_q^{(i)} B_q^{(j)} \right).$$

括号中的每一项都是 INT8×INT8→INT32 GEMM, 外面的 scale 和不同 pair 之间的求和使用 FP64. 因为 A 有 S 个分量、 B 也有 S 个分量, 所以完整展开需要 S^2 个 split-pair.

1.2 初始实现与显存溢出

最初的代码为每个 pair 保存一张完整的 INT32 输出矩阵. 设 $E_C = MN$, 显存需求为

$$\text{INT32 memory} = 4S^2 E_C \text{ bytes.}$$

当 $M = N = 8192$ 时,

$$E_C = 8192^2 = 67,108,864.$$

以 $S = 6$ 为例, 仅中间结果就需要

$$4 \times 6^2 \times 67,108,864 \approx 9.66 \text{ GiB.}$$

MIG 实例只有 10 GiB, 还要同时放置 FP64 输入、输出、INT8 split 和 cuBLAS workspace, 因此在 `splits=6` 时直接触发 `cudaErrorMemoryAllocation`. 这也是本实验最先需要解决的问题.

2 优化过程

2.1 融合多级量化

baseline 每个 split 都启动一次 kernel: 读取 residual, 生成一个 q_i , 再把新的 residual 写回. 这样同一个 FP64 输入会被完整读取 S 次.

修改后的 `quantize_all_splits_kernel` 让一个线程完成同一元素的所有 split. 线程只从输入读取一次, 然后把 residual 和当前 scale 保存在寄存器中循环更新, 最后把 $q_0, q_1, \dots, q_{S-1}$ 写到连续的 split storage 中.

对于 A, B 两个矩阵, 量化 kernel 启动次数从 $2S$ 降为 2. FP64 输入读取量也从约 $2SE$ 降为 $2E$, 代价只是每个元素需要多次寄存器计算和多次 INT8 写出.

2.2 复用 INT32 scratch

不再为所有 pair 分配输出, 而是只保留一个小 batch 的 INT32 矩阵. 一个 batch 中的 GEMM 依次写入不同 scratch 槽位, 随后由重组 kernel 统一计算

$$C_{mn} \leftarrow C_{mn} + \sum_{p=0}^{B-1} \alpha_p T_p[m, n], \quad \alpha_p = s_i^A s_j^B.$$

这样中间显存从 $O(S^2 MN)$ 降为 $O(BMN)$. 在 8192^2 情况下, 一个 INT32 矩阵约为 256 MiB, 8 个槽位约为 2 GiB, 能够和其他缓冲区一起放入 10 GiB MIG 实例.

批量重组还减少了对 C 的重复读改写. 原来每个 pair 都要读取和写回一次 FP64 C ; 现在每个 batch 只需要一次. 以 $S = 8$ 为例, pair 数从 64 个变为若干个 batch, 显存访问和 kernel launch 都明显减少.

2.3 workspace 缓存

benchmark 对相同矩阵会重复运行多次. 如果每次都重新计算最大值、量化和分配显存, 计时中会混入大量固定开销.

因此在 `my_int8_fp64.cu` 中增加持久化 workspace, 并用 (M, N, K, S, dA, dB) 判断是否可以复用. 缓存内容包括:

- A 、 B 的全部 INT8 split;
- host/device pair scale;
- 批量 INT32 scratch;
- split pair 的索引.

同一组输入的后续迭代只需要清零输出并执行 GEMM, 避免重复的 max-abs reduction、量化和 `cudaMalloc/cudaFree`. 这一步对 4096³ 的端到端时间改善尤其明显.

2.4 pair 裁剪实验

因为

$$s_i^A s_j^B \propto 254^{-(i+j)},$$

所以可以优先保留 $i + j$ 较小的 pair. 实际测试得到的现象是:

- 2-split 只保留 3 个 pair, L2 相对误差约 2.7×10^{-5} , 评测可以接受;
- 4-split 保留 10 个 pair 时, 误差约 5.4×10^{-10} ;
- 6/8-split 如果也只保留 10 个 pair, 误差会升到约 5.4×10^{-10} , 评测会判错.

这说明高 split 数对应更严格的精度阈值, 不能仅凭 scale 很小就把 pair 全部删掉. pair 裁剪需要同时观察 GFLOPS 和误差, 否则可能出现“计算更快但得分为零”的情况.

3 最终 fast path

手写 INT8 路径已经解决 OOM, 也通过了 scratch 和缓存优化, 但仍然需要多次 `cublasGemmEx`, 并且部分积需要在 HBM 和重组 kernel 之间往返. 对于 100 分目标, 这些固定开销仍然太大.

CUDA 13 的 cuBLAS 已经提供 Hopper fixed-point emulation. 最终在 `gemm_my_int8_fp64` 中加入 fast path:

- 持久化申请 2 GiB workspace;
- 设置 `CUBLAS_FP64_EMULATED_FIXEDPOINT_MATH`;
- 使用 eager emulation strategy 和 fixed mantissa control;
- 设置 `max_mantissa_bits = min(8S, 55)`;
- 直接调用一次 `cublasGemmEx`, 输入输出仍为 FP64.

这个接口内部完成定点分解、INT8 Tensor Core GEMM 和 FP64 epilogue. 相比手写版本, 它可以在库内部复用 tile, 减少完整 INT32 部分积落到 HBM 的次数, 并把多个阶段的调度开销合并掉. 若 workspace 申请失败, 代码仍然回退到前面实现的手写路径.

4 显存流量分析

以 8192³、 $S = 8$ 为例:

$$8192^2 = 67,108,864,$$

所以一个 FP64 输出矩阵约为 512 MiB, 一个 INT32 部分积约为 256 MiB.

朴素实现有 64 个 pair，仅 INT32 结果的写出和重组读回就约为

$$64 \times 256 \text{ MiB} \approx 16 \text{ GiB}.$$

这还没有计算 64 次对 FP64 C 的读改写. 批量 scratch 把同时存活的 INT32 数据限制在约 2 GiB；workspace 缓存则消除了重复量化和重复分配. 最终 cuBLAS emulation 将 split 调度和 epilogue 放到库内部，进一步缩短了中间数据的生命周期.